

PRACTICAL ACTIVITY 2: Comparing ECB and CBC Modes in AES

Objective:

To implement AES encryption using both ECB (Electronic Codebook) and CBC (Cipher Block Chaining) modes in Python and observe how each mode handles repeating plaintext blocks.

Tools & Setup:

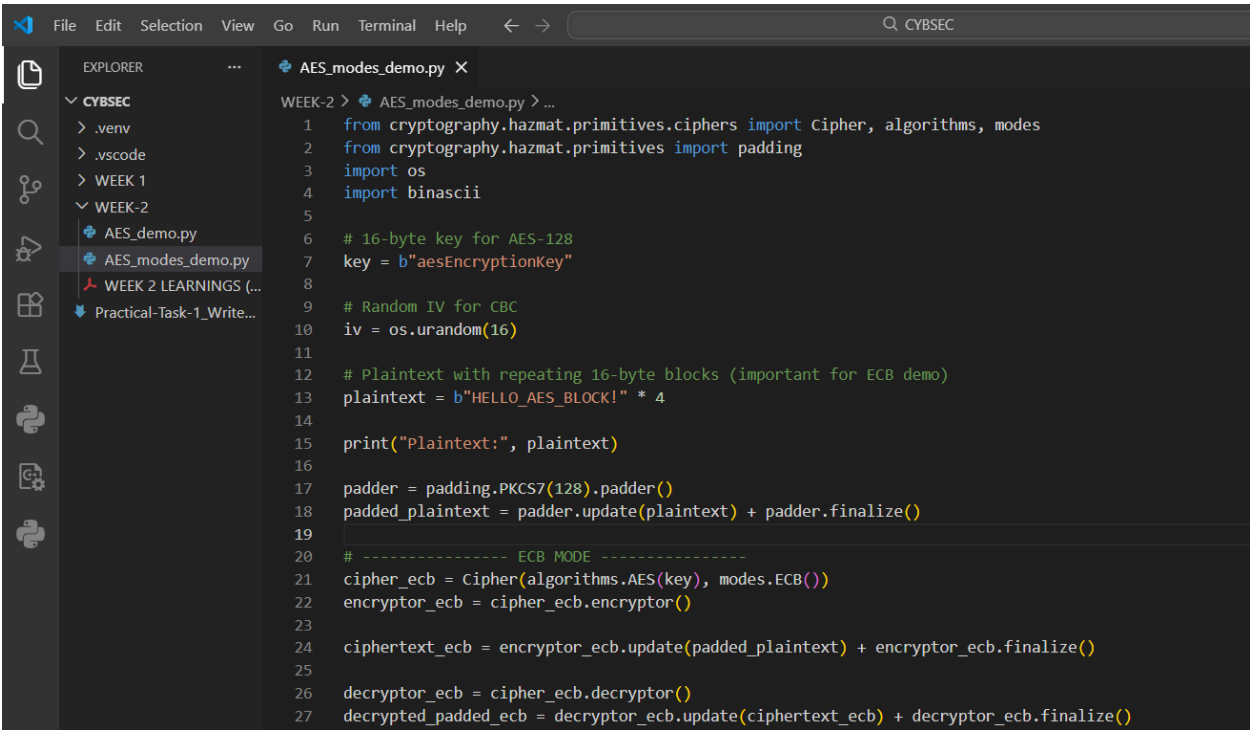
LANGUAGE: Python

LIBRARY: cryptography

ENVIRONMENT: Local machine using a Python script (AES_modes_demo.py)

PROCEDURE:

I began by preparing the Python environment and creating a script named **AES_modes_demo.py**. In this script, I defined a plaintext message containing **repeating 16-byte blocks: HELLO_AES_BLOCK!**. This block was repeated four times to create the full plaintext: **HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!**



```
WEEK-2 > AES_modes_demo.py > ...
1  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
2  from cryptography.hazmat.primitives import padding
3  import os
4  import binascii
5
6  # 16-byte key for AES-128
7  key = b"aesEncryptionKey"
8
9  # Random IV for CBC
10 iv = os.urandom(16)
11
12 # Plaintext with repeating 16-byte blocks (important for ECB demo)
13 plaintext = b"HELLO_AES_BLOCK!" * 4
14
15 print("Plaintext:", plaintext)
16
17 padder = padding.PKCS7(128).padder()
18 padded_plaintext = padder.update(plaintext) + padder.finalize()
19
20 # ----- ECB MODE -----
21 cipher_ecb = Cipher(algorithms.AES(key), modes.ECB())
22 encryptor_ecb = cipher_ecb.encryptor()
23
24 ciphertext_ecb = encryptor_ecb.update(padded_plaintext) + encryptor_ecb.finalize()
25
26 decryptor_ecb = cipher_ecb.decryptor()
27 decrypted_padded_ecb = decryptor_ecb.update(ciphertext_ecb) + decryptor_ecb.finalize()
```

The repeated structure was intentionally chosen because **AES operates on 16-byte blocks**, and repeating blocks help demonstrate how different encryption modes handle patterns.

Before encryption, the plaintext was padded using **PKCS7 padding** to ensure the data length is compatible with AES block requirements.

ECB Mode:

First, I implemented AES encryption using **ECB (Electronic Codebook) mode**. In this mode, each plaintext block is encrypted **independently using the same key**. Because the plaintext blocks in my test were identical, the resulting ciphertext blocks were also identical.

This behavior can be observed in the ciphertext output, where the same hexadecimal block appears repeatedly: 6bb1f1f44091e92f4a17ca94817c5344

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\CYBSEC> CD week-2
PS C:\CYBSEC\WEEK-2> python AES_modes_demo.py
Plaintext: b'HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!'

ECB Ciphertext: b'6bb1f1f44091e92f4a17ca94817c53446bb1f1f44091e92f4a17ca94817c53446bb1f1f44091e92f4a17ca94817c5344c43037a758f93a86d0bf8f0862e7b2f'
ECB Decrypted: b'HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!'

CBC Ciphertext: b'1a17afd7ebc5a8cdbaa56d17d3049887aed76f291935469f77da650c2132c6872ab3ab6ef3be99a3a419d60a68e6978cd86e7774b06611cbbecf424e31f9432bdd394b1f1f5e648cb6528ff99ef70428'
CBC Decrypted: b'HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!'
PS C:\CYBSEC\WEEK-2>
```

After decryption and removal of padding, the original plaintext was successfully recovered.

CBC Mode:

Next, I implemented **CBC (Cipher Block Chaining) mode**. In this mode, a **random initialization vector (IV)** is generated and used during encryption. Each plaintext block is XORed with the previous ciphertext block before encryption, creating a chain dependency between blocks.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\CYBSEC> CD week-2
PS C:\CYBSEC\WEEK-2> python AES_modes_demo.py
Plaintext: b'HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!'

ECB Ciphertext: b'6bb1f1f44091e92f4a17ca94817c53446bb1f1f44091e92f4a17ca94817c53446bb1f1f44091e92f4a17ca94817c5344c43037a758f93a86d0bf8f0862e7b2f'
ECB Decrypted: b'HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!'

CBC Ciphertext: b'1a17afd7ebc5a8cdbaa56d17d3049887aed76f291935469f77da650c2132c6872ab3ab6ef3be99a3a419d60a68e6978cd86e7774b06611cbbecf424e31f9432bdd394b1f1f5e648cb6528ff99ef70428'
CBC Decrypted: b'HELLO_AES_BLOCK!HELLO_AES_BLOCK!HELLO_AES_BLOCK!'
PS C:\CYBSEC\WEEK-2>
```

Because of this chaining process, even though the plaintext blocks are identical, the resulting ciphertext blocks are **different from each other**. The ciphertext output therefore appears more random and does not reveal repeating patterns.

Decryption using the same key and IV successfully restored the original plaintext.

Comparison:

Both encryption modes correctly encrypted and decrypted the plaintext. However, their behavior differed significantly:

- **ECB Mode:** Identical plaintext blocks produced identical ciphertext blocks.
- **CBC Mode:** Ciphertext blocks differed due to chaining and the use of an initialization vector.

This experiment clearly illustrates how ECB can leak patterns in the original data, while CBC obscures these patterns.

Reflection:

This activity demonstrated that the choice of encryption mode significantly affects the security of encrypted data. Although ECB successfully encrypts and decrypts information, its deterministic nature means that identical plaintext blocks always produce identical ciphertext blocks. This can reveal structural patterns in the data, making ECB unsuitable for encrypting structured information such as images or formatted documents.

In contrast, CBC improves security by introducing block chaining and randomness through an initialization vector. This ensures that even identical plaintext blocks result in different ciphertext blocks, making pattern analysis much more difficult.

Through this activity, I gained a clearer understanding that encryption algorithms alone are not sufficient for security; the mode of operation also plays a critical role. Implementing secure modes like CBC helps protect against pattern leakage and strengthens the confidentiality of encrypted data.